

尼恩：LLM大模型学习圣经PDF的起源

在40岁老架构师 尼恩的**读者交流群**(50+)中，经常性的指导小伙伴们改造简历。

经过尼恩的改造之后，很多小伙伴拿到了一线互联网企业如得物、阿里、滴滴、极兔、有赞、希音、百度、网易、美团的面试机会，拿到了大厂机会。2025年开始，尼恩一直在辅导小伙伴们做 AI 架构面试，很多小伙伴 拿到了 Java + AI 架构offer，比如下面的案例：

[34岁无路可走，一个月翻盘，拿 3个架构offer，靠 Java+AI 逆天改命!!!](#)

[3年 程序媛 被裁，25W-》40W 上岸，逆涨60%。Java+AI 太神了，架构小白 2个月逆天改命](#)

[36岁/失业7个月/彻底绝望。狠卷 3个月 Java+AI，终于逆风翻盘，顺利 上岸](#)

尼恩架构团队，通过 梳理一个《LLM大模型学习圣经》帮助更多的人做LLM架构，拿到年薪100W，这个内容体系包括下面的内容：

- [《Python学习圣经：从0到1精通Python，打好AI基础》](#)
- [《LLM大模型学习圣经：从0到1吃透Transformer技术底座》](#)
- [《LangChain学习圣经：从0到1精通LLM大模型应用开发的基础框架》](#)
- 《LLM大模型学习圣经：从0到1精通RAG架构，基于LLM+RAG构建生产级企业知识库》
- [《SpringCloud + Python 混合微服务架构，打造AI分布式业务应用的技术底层》](#)
- 《LLM大模型学习圣经：从0到1吃透大模型的顶级架构》
- 《LLM 智能体 学习圣经：从0到1吃透 LLM 智能体 的架构 与实操》
- 《LLM 智能体 学习圣经：从0到1吃透 LLM 智能体 的中台 架构 与实操》

而 DeepSeek 很复杂，在辅导小伙伴的面试过程中，发现大家对 DeepSeek 底层原理不理解。

接下来，尼恩用20年的技术内功，用20张图+ 大白话介绍，带大家来穿透 MOE架构。

DeepSeek是什么？

成立于2023年7月17日，由杭州知名量化资管巨头幻方量化创立，即深度求索，幻方量化为DeepSeek 的技术研发提供了强大的硬件支持。

<https://www.deepseek.com/>



DeepSeek-V3 在推理速度上相较历史模型 有大幅提升。 在目前大模型主流榜单中，DeepSeek-V3 在开源模型中位列榜首，与世界上最先进的闭源模型不分伯仲。

DeepSeek-V3 的综合能力

DeepSeek-V3 在推理速度上相较历史模型有了大幅提升。
在目前大模型主流榜单中，DeepSeek-V3 在开源模型中位列榜首，与世界上最先进的闭源模型不分伯仲。

Benchmark (Metric)	DeepSeek V3	DeepSeek V2.5	Qwen2.5	Llama3.1	Claude-3.5	GPT-4o
		0905	72B-Inst	405B-Inst	Sonnet-1022	0513
Architecture	MoE	MoE	Dense	Dense	-	-
# Activated Params	37B	21B	72B	405B	-	-
# Total Params	671B	236B	72B	405B	-	-
MMLU (EM)	88.5	80.6	85.3	88.6	88.3	87.2
MMLU-Redux (EM)	89.1	80.3	85.6	86.2	88.9	88.0
MMLU-Pro (EM)	75.9	66.2	71.6	73.3	78.0	72.6

DeepSeek发展历程

第一阶段：创立与核心技术突破（2023年）

2023年初，DeepSeek由多位来自中国顶尖高校和科技企业的AI专家联合创立。

创始团队认为，AGI的实现需要“更高效的模型架构”与“更低成本的训练方法”。

成立仅3个月后，团队即发布首个开源模型DeepSeek-R1，该模型在自然语言理解任务中以百亿参数量达到千亿级模型的性能，首次验证了“轻量化+高精度”技术路线的可行性。

这一突破迅速吸引投资机构关注，公司完成数亿元天使轮融资。

第二阶段：开源生态与行业落地（2024年）

2024年成为DeepSeek的生态扩张年。

公司推出DeepSeek-1.3B模型，首次在代码生成、多轮对话等复杂任务中超越同等规模的国际开源模型（如Meta的LLaMA），GitHub星标数突破3万。

同时，DeepSeek发布自研的分布式训练框架DeepSpeed-Lite，将大模型训练效率提升40%，并开源全套工具链。

这一阶段，DeepSeek与清华大学、上海人工智能实验室等机构建立联合实验室，推动学术与产业协同创新。

第三阶段：多模态与全球化布局（2025年至今）

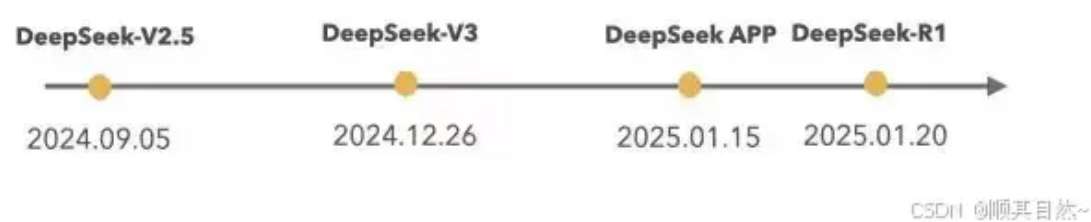
2025年，DeepSeek发布全球首个千亿参数级多模态模型DeepSeek-Vision，支持文本、图像、视频的跨模态推理，在医疗影像分析、工业质检等领域实现商业化落地。

同年，公司与微软Azure达成战略合作，推出企业级AI平台DeepSeek Enterprise，服务金融、制造、教育等行业的500余家客户。

2026年，DeepSeek启动“全球开发者计划”，在硅谷、新加坡设立研发中心，模型下载量突破1000万次，成为GitHub最活跃的AI开源项目之一。

DeepSeek的MOE架构

DeepSeek 相继推出了3个主要的大模型版本，分别是DeepSeek V2.5、DeepSeek V3、DeepSeek-R1。



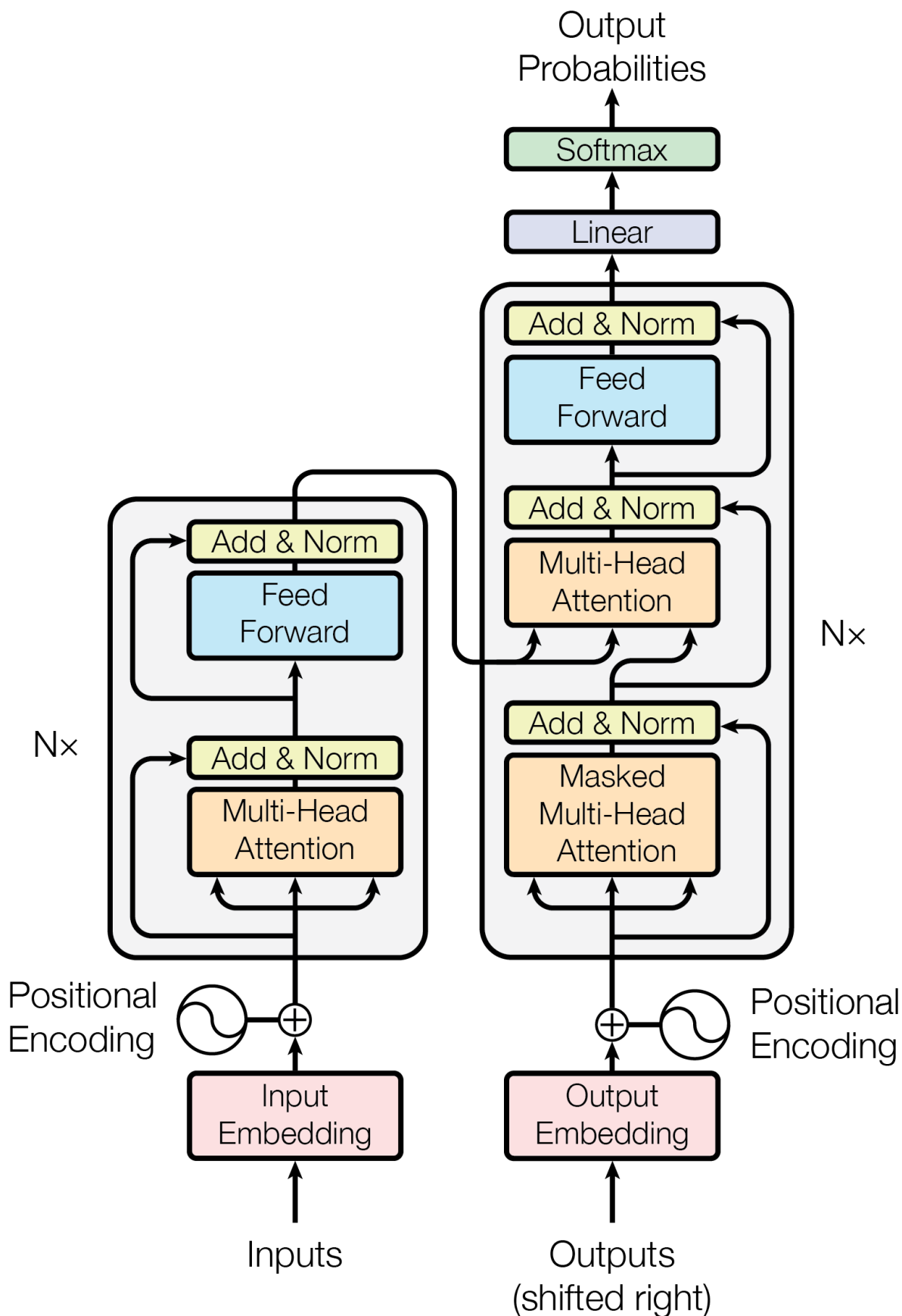
DeepSeek V2.5、DeepSeek V3、DeepSeek-R1 无一例外的都是用了MOE架构。

接下来，尼恩用20年的技术内功，用20张图，正式开始带大家来穿透 MOE架构。

Transformer整体架构

首先，45岁老java架构师尼恩特别说明：
要看懂deepseek 就一定要 回顾 Transformer 的整体结构，要不然 看不懂本文。

Transformer整体架构 如下：



以下的小段内容，来自于尼恩的 PDF 文章《LLM 大模型学习圣经》：

Transformer 抛弃了传统的 RNN 和 CNN， 特点如下：

- 首先它使用了 Attention 机制，将序列中的任意两个位置之间的距离是缩小为一个常量；

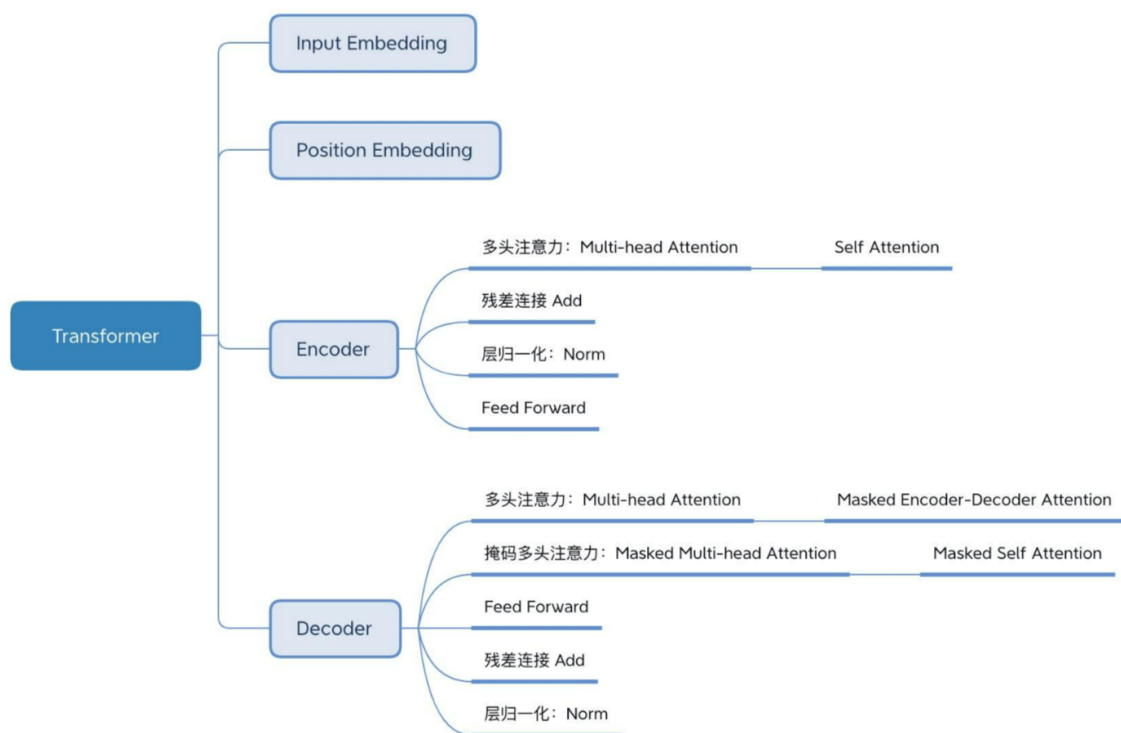
- 其次，Transformer 不是类似RNN的顺序结构，因此具有更好的并行性，是多头注意力机制，符合现有的GPU框架，有效的解决了NLP中棘手的长期依赖问题。

Transformer是一个encoder-decoder的结构，由若干个编码器和解码器堆叠形成。

如上图

- 左侧部分为编码器，由Multi-Head Attention和一个全连接组成，用于将输入语料转化成特征向量。
- 右侧部分是解码器，其输入为编码器的输出以及已经预测的结果，由Masked Multi-Head Attention, Multi-Head Attention以及一个全连接组成，用于输出最后结果的条件概率。

简单梳理一下，大概包括下面的几个部分：



要看懂deepseek，就一定要 回顾 Transformer 的整体结构，要不然，看不懂本文。关于Transformer，请异步 尼恩的PDF 文章《LLM 大模型学习圣经》。

DeepSeek对Transformer架构两大改进

DeepSeek 整体上是Transformer架构，但是进行的架构的优化和改进，两个重要的改进为

- FFN 前馈网络使用的DeepSeekMoE架构
- Attention机制使用MLA架构

另外，与DeepSeek-V2相比，DeepSeek-V3额外引入了一种 负载均衡策略，用于DeepSeekMoE，以减轻因需要保证Expert负载均衡而导致的性能下降。

DeepSeek-V3 模型 网络架构的两大改进

为了在训练和推理时跑得更快、效果更好，DeepSeekV3 自己研发了两个核心技术：

1. **MLA（多头潜在注意力）**
2. **无辅助损失的 MoE 负载均衡策略**

什么是 MLA（多头潜在注意力）？（多角度看问题）

45岁老架构师给大家的直白的解释，MLA 就是 多角度看问题

MLA（多头潜在注意力），用来提升模型理解和生成能力的一种机制。

MLA（多头潜在注意力），可以想象成一个人在分析问题的时候，同时从多个角度看一个问题。

比如看一幅画，可以从颜色、形状、构图等多个角度去分析，这样看得更全面、理解得更深入。

MLA 就是让模型在处理语言时，能从多个“角度”提取信息，从而更好地理解上下文。

什么是无辅助损失的 MoE 负载均衡策略？（部门分工、专人专事）

45岁老架构师给大家的直白的解释，MoE 就是 部门分工、专人专事，不再是一个人打全场。

可以把DeepSeek想象成一个公司：公司里有很多不同部门（比如技术部、市场部、财务部），每个部门都擅长处理特定类型的任务。当有任务来的时候，由“前台”来决定把任务分配给哪个部门处理。

当然，MoE 有时候可能会出现这样的情况：

- 有些专家特别忙，总是被调用；
- 而有些专家却很闲，几乎没活干。

这就需要有一个调度员来合理分配任务，让每个专家都能发挥自己的作用。

DeepSeekV3 使用了一种新的负载均衡方法，不需要额外加复杂的规则，就能让各个“专家”之间的工作量更平均，提高整体效率。

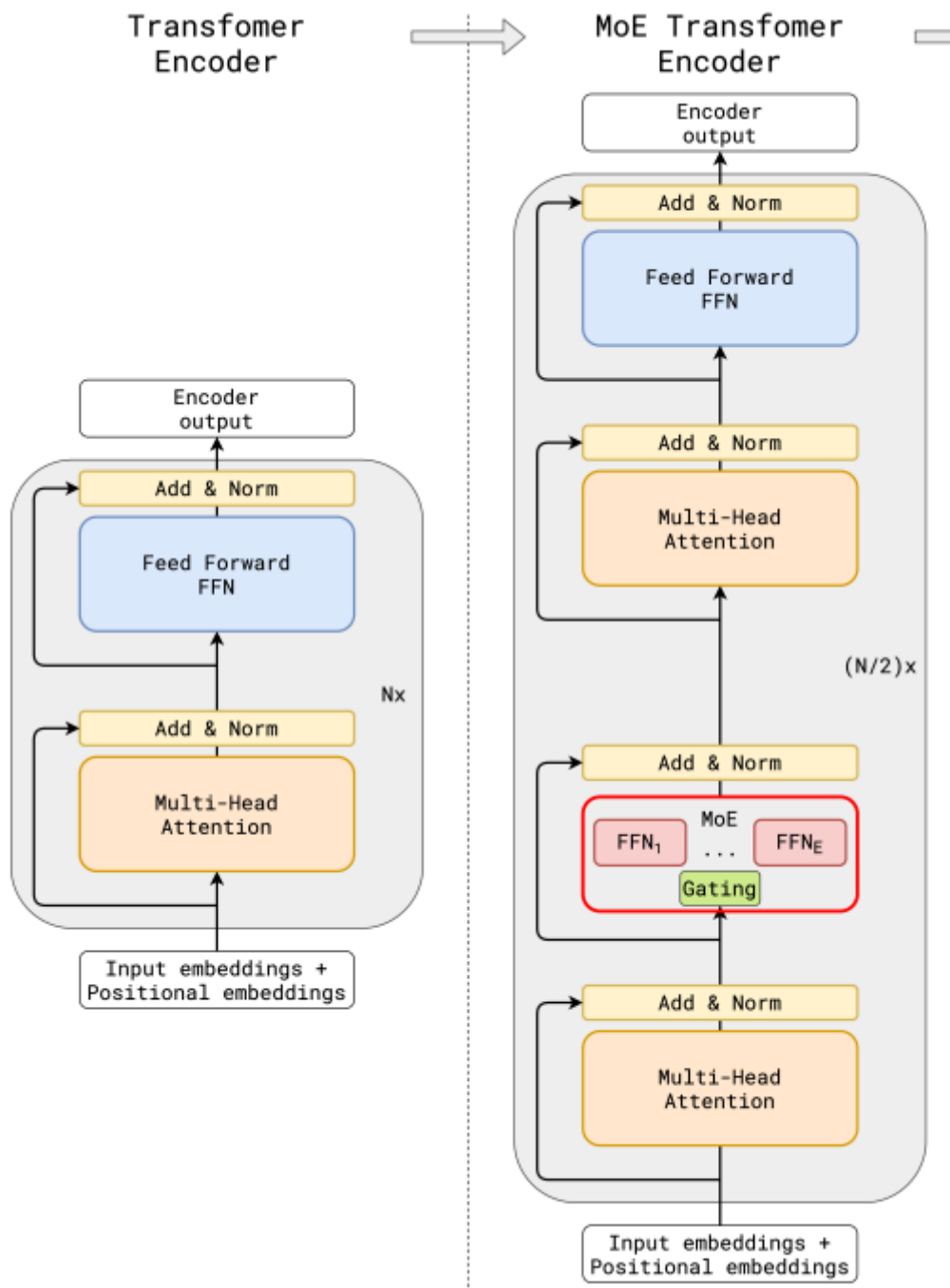
DeepSeekMoE + MLA 注意力机制 的 总结说明：

技术	简单说明
DeepSeekMoE	多个专家模型协同工作，任务分配更智能，训练效率更高
MLA 注意力机制	对 Key 和 Value 压缩存储，减少缓存占用，推理更快

这两项技术让 DeepSeekV3 在保持强大性能的同时，训练和推理都更快了。

基础架构的第一个优化：DeepSeekMoE架构

第一个将MoE架构引入Transformer网络的就是GShard架构了，GShard架构图如下：



与传统大模型架构相比，GShard 架构下 MoE 在数据流转过程中集成了一个专家网络层。

可以看出传统的MoE基本两部分组成：Gating门控网络、稀疏MoE层；

- 稀疏 MoE 层：

这些层代替了传统 Transformer 模型中的前馈网络 (FFN) 层。MoE 层包含若干“专家”(例如 8 个)，每个专家本身是一个独立的神经网络。

在实际应用中，这些专家通常是前馈网络 (FFN)，但它们也可以是更复杂的网络结构，甚至可以是 MoE 层本身，从而形成层级式的 MoE 结构。

- 门控网络或路由：

这个部分用于决定哪些Token被发送到哪个专家。

Token的路由方式是 MoE 使用中的一个关键点，因为路由器由学习的参数组成，并且与网络的其他部分一同进行预训练。

和传统的MoE架构相比，DeepSeekMoE使用更细粒度的专家，并将一些专家隔离为共享专家，减少专家间的知识冗余。



- **门控网络路由策略：**

TopK表示第t个Token和所有路由专家计算出的亲和力分数中K个最高分数的集合，在DeepSeekV3中，使用sigmoid函数计算亲和力分数，然后在所有选择的亲和力分数中应用归一化来生成门控值。

通常在MoE模型的训练过程中，不同专家因为路由策略的因素会导致接收的训练数据分布不均，比如所有的Token都被发送到只有少数几个受欢迎的专家，那么有些专家就可能没有被训练到。

业界通用的解决方案就是引入辅助损失，但是，有时候过大的辅助损失会损害模型性能。

- **无辅助损失的负载均衡策略**

为了在负载均衡和模型性能之间取得更好的平衡，DeepSeek开创了一种无辅助损失的负载均衡策略：为每个专家引入一个偏差项 b_i ，并将其添加到相应的亲和力分数 $S_{i,t}$ 中以确定top-K路由。

具体来说：如果其对应的专家过载，DeepSeek 将偏差项减少 γ ；如果其对应的专家负载不足，DeepSeek将偏差项增加 γ ，其中 γ 是一个称为偏差更新速度的超参数。

门控网络本质上就是一个softmax叠加一个分类网络，那么辅助loss往往就是添加一个惩罚项，对输出过大的 logits 进行惩罚，鼓励模型生成更加适度的 logits 值，防止模型生成过于极端的输出。

DeepSeekV3 是一个非常大的语言模型，它的参数总量达到了 **6710亿**（也就是 671B），但你不用被这个数字吓到。

其实每次处理一句话或者一段文字时，它只用到了其中一小部分参数，大概有 **370亿**（37B）左右在工作。

这背后，靠的就是这个 一个叫 **MoE（Mixture of Experts，专家混合）** 的架构。

直观的说，可以把DeepSeek 想象成一个“大公司”，里面几百个员工（专家），但不是每次任务都让所有人一起干，而是根据任务内容，挑几个最合适的专家来完成。这样既高效又省资源。

第一大架构改进：MoE（专家混合）结构通俗讲解

MoE，全称 **Mixture of Experts**，中文叫“专家混合”。

你可以把它想象成一个公司：公司里有很多不同部门（比如技术部、市场部、财务部），每个部门都擅长处理特定类型的任务。当有任务来的时候，由“前台”来 决定把任务分配给哪个部门处理。

在 MoE 中：

- **各个部门** = 各个“专家” (Expert)
- **“前台”** = “门控网络” (Gating Network)

传统 MoE 的结构

传统 MoE 主要由两个部分组成：

1. 门控网络 (Gating Network)

就像公司的“前台”，它负责判断每个输入的 Token（可以理解为一句话中的一个词）应该交给哪个专家去处理。

2. 稀疏 MoE 层 (Expert Layer)

这一层里有多“专家”，每个专家其实就是一个小型神经网络。

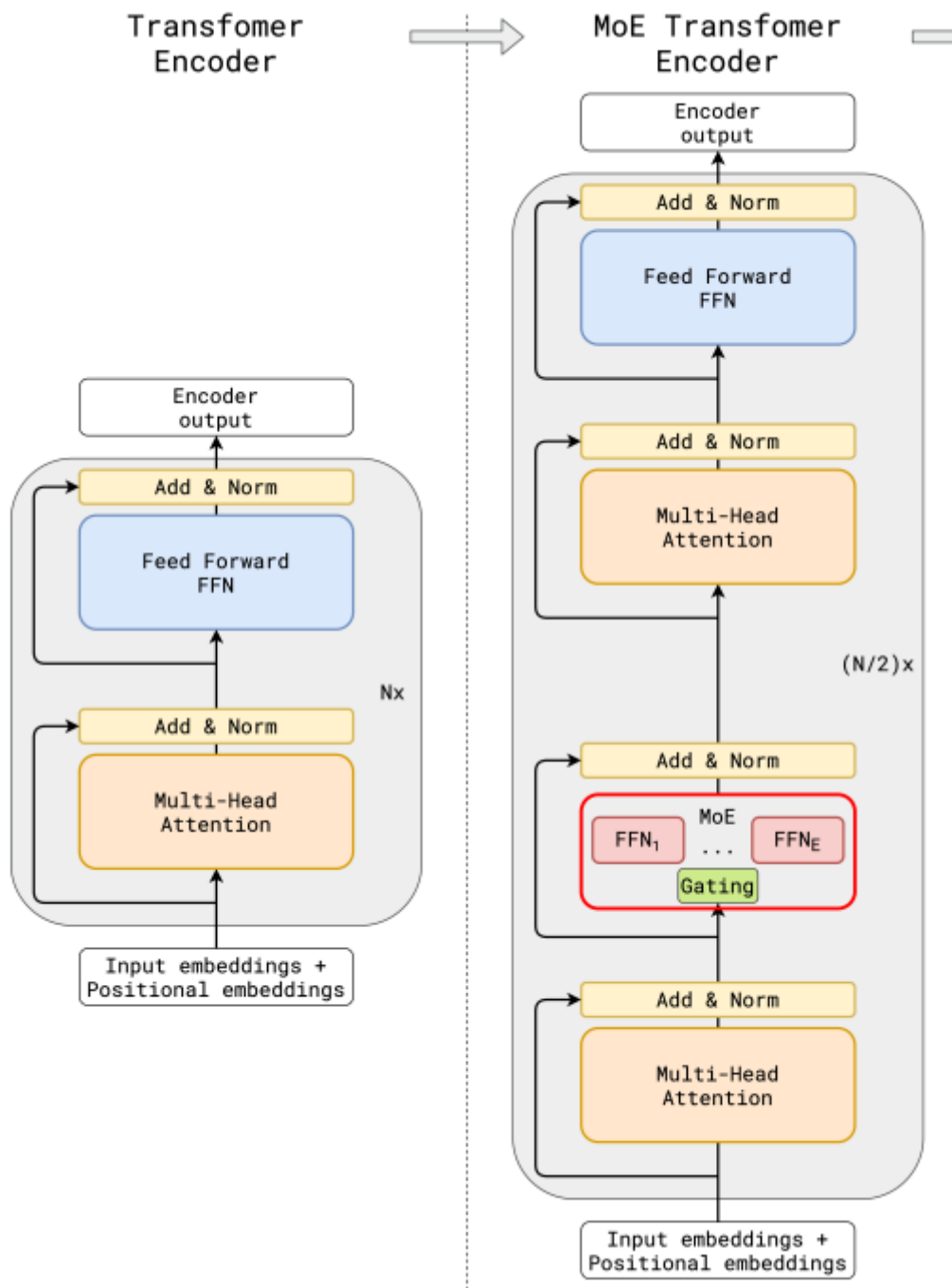
它们只处理被门控网络选中的 Token。

举个例子：

假设你输入一句话：“我今天想买手机”。

门控网络会分析这句话，觉得“买”和“手机”这两个词更适合交给“购物专家”处理，而“我”和“今天”可能更适合交给“日常对话专家”处理。

这样，每个专家只负责处理它擅长的部分，效率更高。



DeepSeekMoE 的创新点

DeepSeekMoE 在传统 MoE 的基础上做了两个重要改进：

- **更细粒度的专家划分**

把专家分得更细，每个专家只专注处理更具体、更细分的任务。

就像把公司部门划分得更细致，比如从“技术部”再分出“前端组”、“后端组”、“AI组”。

- **引入共享专家**

有些任务可能多个专家都能处理，这时候可以引入一个“通用专家”，大家都可以调用它，避免重复劳动，提高效率。



门控网络是怎么工作的？

门控网络就像是一个“智能调度员”，它会为每一个 Token 计算出它和各个专家之间的“匹配度分数”，然后选出最合适的 Top-K 个专家来处理这个 Token。

在 DeepSeekV3 中，这个匹配度是通过 **Sigmoid 函数** 来计算的，然后还会做一个“归一化”处理，确保这些分数加起来是 1，这样就能决定每个专家应该承担多少任务量。

负载均衡问题及解决方法

如果某些专家总是被选中干活，而其他专家一直“闲着”，就会导致训练不均衡，影响整体效果。

通常的做法是加一个“惩罚项”（辅助损失），但这种方法有时会影响模型性能。

DeepSeekV3 提出了一个 **不需要额外惩罚项的负载均衡方法**：

- 给每个专家加一个“偏差项” (b_i)
- 如果某个专家太忙（过载），就减小它的偏差，让它“少干活”
- 如果某个专家太闲（负载不足），就增大它的偏差，让它“多干活”

这样可以动态调整专家的使用频率，达到更好的平衡。

DeepSeekMoE 的核心流程图



第二大架构改进： MLA 多头潜在注意力

为什么注意力机制会影响大模型的效率？

在大模型中，**注意力机制**就像是一个“大脑”，它负责决定在处理一句话或一段内容时，哪些部分更重要，应该多关注一些。

但这个“大脑”有个问题：它会占用很多内存和计算资源，尤其是在模型推理（或者说生成答案）时。

其中，**KV Cache（键值缓存）** 是个大头。

可以把**KV Cache（键值缓存）**理解为模型在生成回答时，为了记住之前已经处理过的内容，需要不断保存一些中间数据。

KV Cache（键值缓存） 数据越多，模型运行得就越慢。

传统的注意力机制叫 **MHA（多头注意力）**，它虽然效果好，但会生成大量的 KV Cache，导致推理变慢。

为了解决这个问题，业界尝试了很多方法，比如：

- **PagedAttention**：像分页管理内存一样管理 KV Cache
- **MQA（多查询注意力）**：减少 Key 和 Value 的数量
- **GQA（分组查询注意力）**：把多个头分组处理，节省资源

但这些方法在性能上，往往不如原生的 MHA。

MLA（多头潜在注意力）是什么？怎么解决这个问题？

DeepSeekV2 提出了一个新方法，叫做 **MLA (Multi-head Latent Attention)** **多头潜在注意力**，在 V3 中也继续使用并优化了它。

MLA 的核心思想是：

对 Key 和 Value 做压缩，减少缓存占用，同时尽量不损失性能。

可以这样理解：就像我们压缩图片或视频一样，先把数据“缩小”存起来，等要用的时候再“放大”还原回来。这样

举个例子：低秩压缩是怎么回事？

假设你有一个表格，是 4 行 5 列的数据（4x5 的矩阵），看起来很大。

但其实，这个表格里数据是有规律的，可以用两个小表格来表示：

- 一个 4 行 2 列的表格
- 一个 2 行 5 列的表格

把这两个小表格相乘，就能还原出原来的 4x5 表格。

这就是所谓的 **低秩压缩**，也就是用更小的数据来表示原来的大数据。

MLA 就是用了这个思路，把 Key 和 Value 向量进行压缩，减少存储空间。

MLA（多头潜在注意力）的计算流程是怎样的？

MLA 的整个流程可以分为几个步骤：

1. **输入 Token h_t** ：模型接收到当前要处理的词或字符
2. **压缩处理**：用一个矩阵 W^{DKV} 把 h_t 压缩成一个小向量 c_t^{KV} ，用于缓存
3. **恢复处理**：在需要的时候，用 W^{UK} 和 W^{UV} 把 c_t^{KV} 还原成原始的 Key 和 Value
4. **注意力计算**：用还原后的 Key 和 Value 做标准的注意力计算
5. **输出结果**：得到最终的输出

$$\begin{aligned} c_t^{KV} &= W^{DKV} h_t \\ k_t^C &= W^{UK} c_t^{KV}, \\ v_t^C &= W^{UV} c_t^{KV} \end{aligned}$$

CSDN @顺其自然~

MLA 的核心流程图



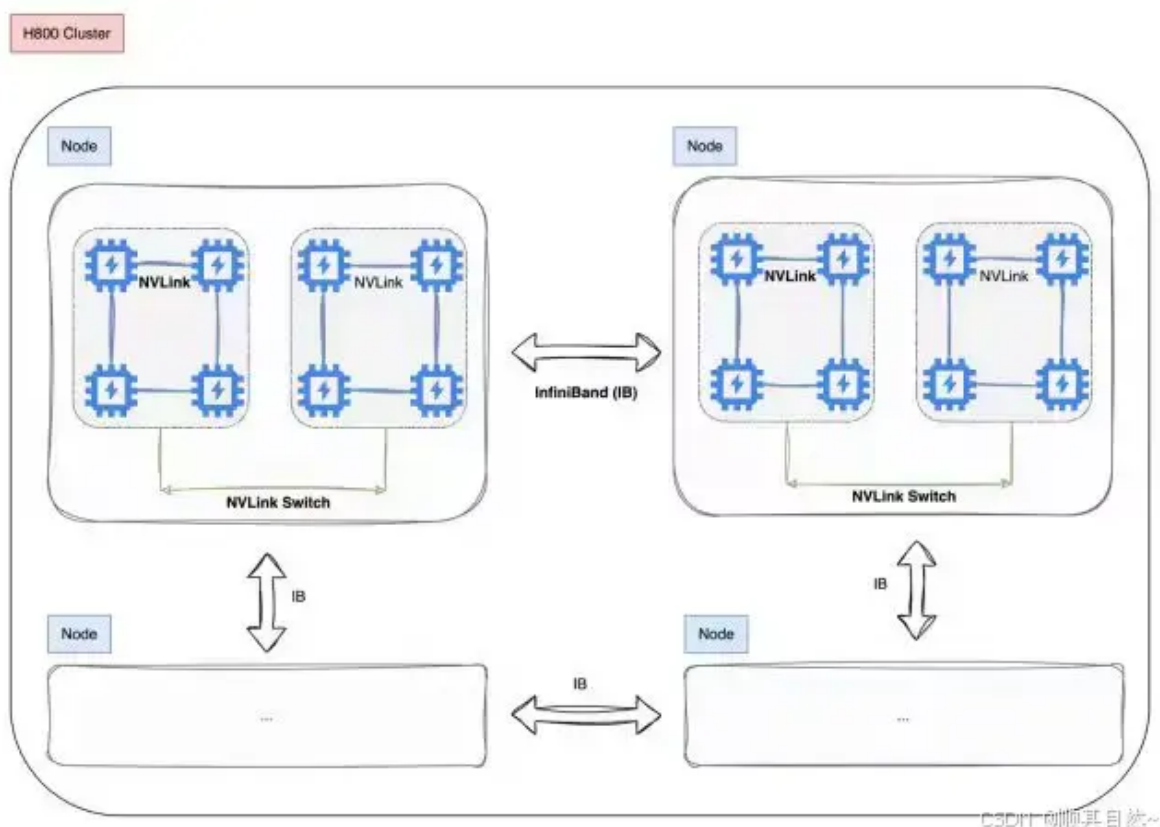
训练推理的四大核心改进

DeepSeek-V3在一个配备了2048个NVIDIA H800 GPU的集群上进行训练，使用的是自研的HAI-LLM框架，框架实现了四种并行训练方式：ZeRO 支持的数据并行、流水线并行、张量切片模型并行和序列并行。

这种并行能力支持不同工作负载的需求，可以支持数万亿规模的超大模型并扩展到数千个 GPU，同时还自研了一些配套的高性能算子haiscale，可以帮助 HAI-LLM 极大优化大模型训练的显存效率和计算效率。

DeepSeek 训练推理的四大核心改进

- 训练框架 HAI-LLM
- 核心算法DualPipe-创新流水线并行算法
- 用于FP8训练的混合精度框架
- 多 Token 预测（MTP）架构演进



训推第一个改进：自研的 训练框架 HAI-LLM

DeepSeek-V3 是在一个非常强大的服务器集群上训练出来的，这个集群配备了 **2048 块 NVIDIA H800 显卡**，相当于一个超级计算机。

它用的是 DeepSeek 自己开发的一套训练框架，叫做 **HAI-LLM**。

这套 **HAI-LLM** 框架专门用来训练超大规模的语言模型，比如 DeepSeek-V3 这种拥有万亿参数的大模型。

这个框架之所以强大，是因为它支持四种不同的并行方式，能高效地利用这么多显卡来一起工作：

1. 数据并行 (ZeRO) :

把训练的数据分成很多小份，每块显卡处理一小份，最后再把结果合起来。就像一群人一起做同一道数学题，每人算一部分，最后拼成完整答案。

2. 流水线并行:

把整个模型拆成几段，像工厂里的流水线一样，一块显卡处理完一段后传给下一块，这样效率更高。

3. 张量切片模型并行:

把模型中一些特别大的计算任务切成小块，分配到不同显卡上同时运算，加快训练速度。

4. 序列并行:

对输入文本的不同部分进行并行处理，特别适合处理长文章或对话。

这几种方法配合使用，让 HAI-LLM 能够轻松应对各种训练任务，不管是小模型还是超大模型都能搞定，还能扩展到上千块显卡一起工作。

为了进一步提升训练效率，团队还开发了一个高性能算子工具包，叫 **haiscale**。

haiscale 工具包 可以更高效地管理显存和计算资源，就像是给训练过程装上了“加速器”，让整个训练更快、更省资源。



训推第2个改进：创新的流水线并行算法 DualPipe

实现多个维度的并行

DualPipe 是创新的流水线并行算法，实现多个维度的并行

DeepSeek-V3 在训练大模型时，用了很多方法来加快速度、节省资源。

主要包括三种策略：

- 流水线并行
- 专家并行
- 数据并行

1. 把模型拆成段，分给不同 GPU (流水线并行)

就像把一条生产线分成多个工序，每个 GPU 负责一个阶段。DeepSeek 把整个模型分成了 **16 段**，每段由不同的 GPU 来计算。

这样做的好处是：GPU 可以同时工作，效率更高。

2. 多个“专家”一起干活 (专家并行)

在一些大模型中，会设计很多“专家”模块，每个专家负责一部分任务。

DeepSeek 使用了 **8 个服务器节点**，每个节点有 **8 个 GPU**，总共可以运行 **64 个专家模块**。

你可以想象成：一个工厂里安排了 64 个老师傅，每人专攻一项技能，大家分工合作，效率翻倍。

3. 数据也分开跑 (数据并行)

训练时需要大量数据，如果都放在一个 GPU 上，内存会爆掉。

DeepSeek 使用了一种叫 **ZeRO-1 的技术**，把数据切开，分给各个设备去算，减少单个 GPU 的负担。

DualPipe：让 GPU 更忙起来

传统方式的问题

以前的流水线并行有个毛病：当一个阶段算完后，下一个阶段的 GPU 得等着它传数据才能开始干活。这段时间就浪费了，我们叫它“空转时间”。

这就像工厂里的传送带，前一道工序没做完，下一道工序就得干等，效率低。

DualPipe 的改进

DeepSeek 提出了一个叫 **DualPipe** 的技术，它能更好地安排任务顺序，让计算和通信尽可能同时进行，减少等待时间。

DualPipe 技术的两大特点：

- i.通信计算重叠优化
- ii.跨节点全对全通信

可以看出DeepSeek在PP这块，做了大量的通信计算重叠优化，从技术报告中看出，即使是细粒度的all-all专家通信，all-all的通信开销几乎为0。



DualPipe 是怎么做到 通信计算重叠优化？

它把每个计算块分成四个部分：

1. **注意力计算：**
模型理解上下文关系；
2. **All-to-all调度：**
把数据分配到不同 GPU；
3. **MLP计算：**
做复杂的数学运算；
4. **All-to-all组合：**
把结果再收回来。

举个例子，假设有两个计算块 A 和 B：

- 当 A 在做前向计算时，B 同时在做反向传播的通信；
- 等 A 前向算完后，开始通信；这时 B 开始它的前向计算。

这种“你算我传、我算你传”的方式，让 GPU 几乎没有空闲时间。

实际效果如何？

即使是非常频繁的数据交换（比如每个词都要和其他 GPU 打交道），也能几乎不耽误整体进度。通信开销被“藏”起来了。

图示：DualPipe 如何重叠计算与通信



什么是“通信与计算重叠”？

打个比方：你在厨房做饭，一边炒菜（计算），一边等水烧开（通信）。如果你等水开了再炒菜，就浪费了时间。但如果炒菜的同时烧水，就能省时间。

这就是“通信与计算重叠”的原理。

流水线中的“气泡”问题

在传统的流水线训练中，前面的层还没算完，后面的 GPU 就只能干等，形成“气泡”——也就是空闲时间。

如下图所示，灰色区域就是 GPU 空转的时间：



而 DualPipe 正是通过合理安排任务，把这些“气泡”变小甚至消除，从而提高整体训练效率。

DualPipe 跨节点“全对全”通信架构演进

DeepSeek 在训练大模型时，特别优化了 GPU 之间的数据交换方式，尤其是跨节点之间的“all-to-all”通信（即每个 GPU 都要和其他所有 GPU 交换数据）。

这种优化让模型在大规模扩展时依然保持高效。

网络架构设计

- **节点内部通信**：使用 NVLink，就像 GPU 之间的“高速内网”，速度快、延迟低。
- **节点之间通信**：使用 InfiniBand（简称 IB），就像连接不同大楼之间的“高速公路”，适合远距离高速传输。
- **通信调度限制**：每个 token 最多只分配到 4 个节点，避免网络负担过重。

warp 专业化技术（任务分工）

GPU 是并行计算的高手，它内部有很多小任务单元，叫做 warp。

DeepSeek 利用这些 warp 做了“任务分工”，就像让不同的员工负责不同的工作环节：

- 一部分 warp 负责把数据通过 IB（InfiniBand）发送出去

- 一部分负责把 IB (InfiniBand) 收到的数据转给 NVLink
- 一部分专门接收 NVLink 传来的数据

这些任务是**同时进行的**，而且 warp 的数量会根据任务多少自动调整，确保通信顺畅、不卡顿。

合并结果时的优化

在最后把结果汇总时，也采用了类似的分工：

- 一部分 warp 发送 NVLink 数据
- 一部分负责把 NVLink 数据转发并累加到 IB
- 一部分接收 IB 数据并进行整合

同样是多个 warp 并行处理，动态调整资源，保证效率。

实际效果

通过这种“分工+并行”的方式，IB 和 NVLink 的通信几乎可以**同时完成**，不会互相等待，也就没有额外的延迟。

带来的好处包括：

- 每个 token 平均可以使用 3.2 个专家（节点）
- 整体最多可扩展到 13 个专家（4个节点 × 3.2）
- 即使现在只用了 8 个专家，未来也能轻松扩容，通信成本不会增加

MoE（混合专家模型）下的“all-to-all”通信 核心流程

DeepSeek-V3 使用的是 **混合专家模型（MoE）**，这是一种“按需分配”的模型结构。

具体配置如下：

- **1个共享专家**：所有 token 都会用到，就像一个通用服务窗口
- **256个路由专家**：每个 token 根据自己的需求，选择其中的 8 个专家来处理自己

可以想象成一个“智能快递系统”：每个包裹（token）会自动选择最适合的几个快递员（专家）来运输，既高效又灵活。

核心流程图



流程说明：

- 先做注意力计算（A）
- 然后进行 all-to-all 数据调度（B）
- 接着执行 MLP 计算（C）
- 再次进行 all-to-all 数据组合（D）
- 最后进入下一块计算（E）

训推第3大改进：从单精度 到 高低混合的 精度框架 演进

DeepSeek 使用了 高精度（BF16/FP32）+ 低精度（FP8）混合的 精度框架，这 是一种通过动态组合不同精度格式（如FP8、BF16、FP32）来优化大模型训练效率的技术方案

- 在矩阵乘法（GEMM）、梯度计算等密集型操作中使用FP8格式，显存占用仅为FP16的50%，理论计算速度提升2倍
- 对数值敏感的组件（如归一化层、注意力机制）仍采用BF16/FP32计算，确保训练稳定性

尼恩备注：DeepSeek 做了速度和精度的平衡，做了 速度和 精度 的最优取舍。

通过 精度解耦，将 低精度的FP8格式 用于通信和存储 密集型的 环节（如权重传输），而在 优化器状态等关键参数处理环节，则使用高精度的BF16/FP32格式，减少量化误差累积。

三大主流的数据精度格式 的对比

FP8的全称为**8位浮点数**（8-bit Floating Point），其标准英文名称为 **Float8** 或完整表述 **Floating Point 8**。该名称直接体现了其核心特性：

- “**FP**” 代表浮点数（Floating Point），区别于整型等数据类型；
- “**8**” 指代总存储位数为8比特，显著低于FP16（16位）和FP32（32位）的存储需求

FP8、BF16和FP32是目前深度学习领域主流的数据精度格式，它们在存储结构、数值范围、精度表现和应用场景上存在显著差异。

FP8、BF16和FP32 核心对比如下：

精度格式	总位数/字节	符号位	指数位	尾数位	数值范围	典型精度特点
FP32	32位 (4字节)	1位	8位	23位	$\pm 3.4 \times 10^{38}$	高精度，无显著精度损失
BF16	16位 (2字节)	1位	8位	7位	$\pm 3.39 \times 10^{38}$	指数范围同FP32，尾数精度较低
FP8	8位 (1字节)	1位	4-5位¹	2-3位¹	± 448 (E4M3)	精度损失明显，需动态缩放

1、内存与计算效率

- FP32**：占用资源最多，计算速度最慢，但精度最稳定；
- BF16/FP16**：内存占用为FP32的50%，计算速度提升2-3倍；
- FP8**：内存仅FP16的50%，理论算力翻倍，适合高吞吐场景。

2、数值稳定性

- FP32**：几乎无溢出/下溢风险，适合科学计算和梯度累积；
- BF16**：指数范围与FP32一致，避免训练中梯度溢出（优于FP16）；
- FP8**：动态范围窄，需配合损失缩放技术。

低精度训练中的策略：哪些用 FP8？哪些不用？

在训练模型时，并不是所有部分都使用 FP8 这种低精度的数据格式。这样做是为了在加快训练速度的同时，不让模型的效果下降太多。

FP8 是一种低精度数据格式，虽然节省空间、速度快，但精度低，容易出错。所以 Deepseek 挑着用。

哪些计算适合用 FP8？哪些不适合？

- ☒ **适合用 FP8 的：**

主要是那些计算量大的操作，比如矩阵乘法（GEMM）。这些操作是训练中最耗时间的部分，用 FP8 可以明显提升效率。

- ☒ **不适合用 FP8 的：**

一些对数值特别敏感的操作，比如归一化、注意力机制、Embedding 层等。如果这些模块用了 FP8，可能会导致数值不稳定，影响训练结果。

- ☒ **折中处理的：**

还有一些轻量级操作，虽然也可以用 FP8，但为了稳妥起见，还是保留了更高精度（如 BF16 或 FP32），以确保整体训练过程更稳定。

具体保留高精度的模块包括：

- Embedding 层（词向量）
- 输出头（最后一层预测）
- MoE 门控模块（决定哪个专家网络参与计算）
- Normalization 算子（如 LayerNorm）
- Attention 模块（注意力机制）

如何在低精度下保持训练质量？

1. 更细粒度地做量化

- 对于**激活值 (activation)**：

我们在 token 维度上做 group-wise 的量化，每个 group 是 1×128 大小。

就像把一大块数据切成小块，每一块单独处理，这样能更好地适应不同位置的数据变化。

- 对于**权重 (weight)**：

采用 block-wise 的方式，每个 block 是 128×128 大小。

这种方式可以在不牺牲太多性能的前提下，提高量化精度。

打个比方：这就像切蛋糕，大刀阔斧容易出错，切成小块慢慢调整，更容易做到均匀和准确。

2. 提高累加计算的精度

虽然矩阵乘法是在 FP8 下进行的，但在累加过程中，我们会把中间结果从 TensorCore 转移到 CUDA Core 上，使用 FP32 来做累加。

这就像是你用计算器算账，虽然输入的是整数，但内部用浮点数来避免误差累积。

高低混合的精度框架 核心流程图



如何提高低精度训练精度？

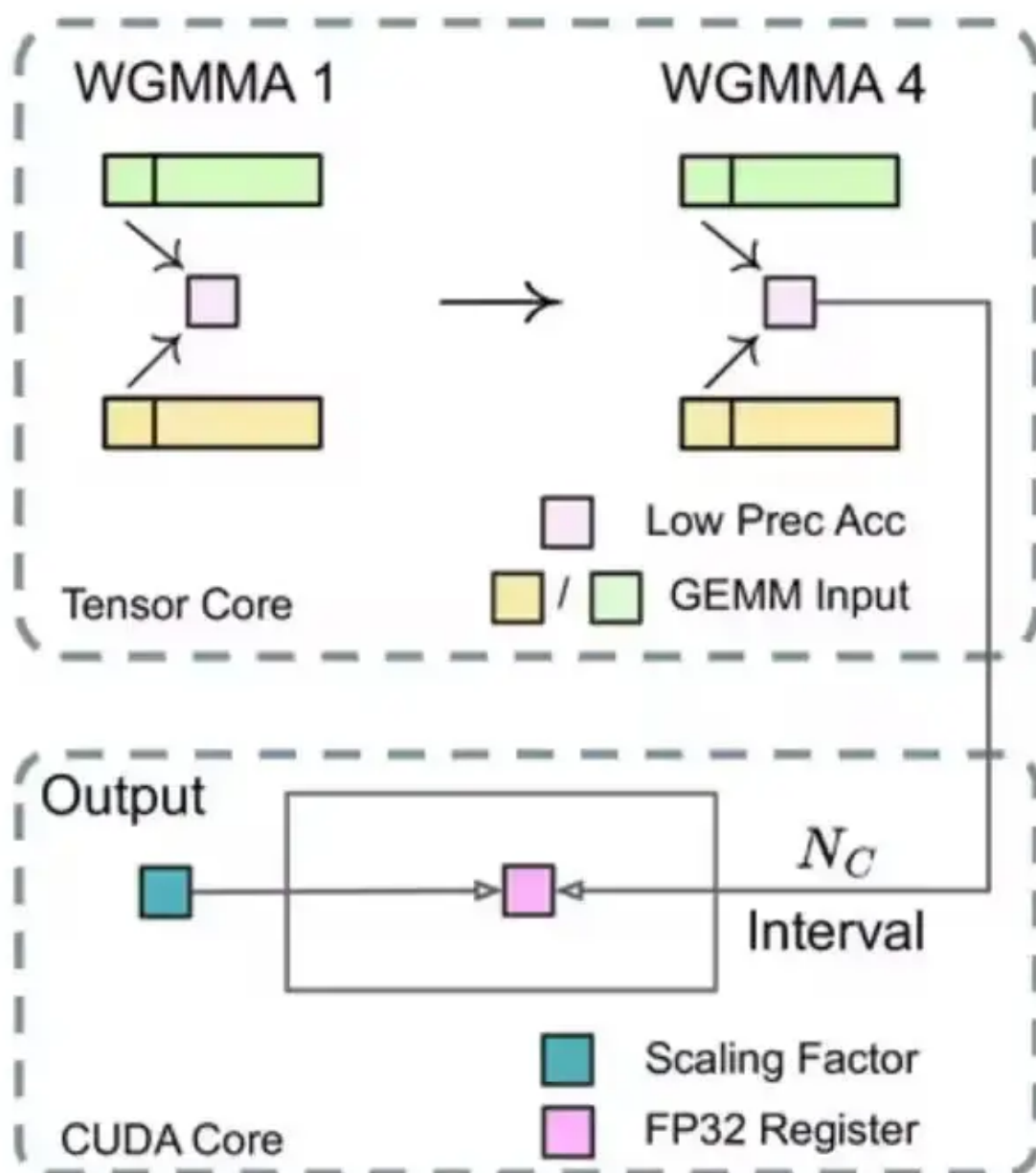
■细粒度量化

对激活，在token维度采用group-wise的量化(1 128)；对权重，采用128 128的block-wise量化



■提高累加精度

在 TensorCore 上执行矩阵 MMA（矩阵乘法累加）操作时，每当累加达到一个间隔时，这些部分结果会被传输到 CUDA Cores 上的 FP32 寄存器中，并在那里进行FP32 精度的累加计算。



CSDN @顺其自然~

总之：Deepseek 是从单精度到高低混合的精度框架演进

DeepSeek在深度学习训练框架中关于数值精度的技术演进路径，其核心是从单一高精度（如FP32）向高低精度混合计算的转变过程。

具体分为三个阶段：

1、单精度阶段（FP32主导）

早期训练完全依赖FP32格式，虽能保证数值稳定性，但计算效率低且显存占用高（如ResNet50需7.8GB显存）。

2、混合精度过渡（FP16+FP32）

引入自动混合精度（AMP）技术，关键改进包括：

- 前向/反向计算使用FP16加速
- 权重和梯度更新保留FP32精度
- 动态损失缩放（Loss Scaling）解决FP16梯度下溢问题。

3、高低混合架构（FP8/BF16+FP32）

最新框架实现更细粒度的精度分层：

- **计算层**：矩阵乘法等密集型操作使用FP8（E4M3/E5M2格式），显存占用降低50%
- **敏感模块**：层归一化、梯度累积等保留BF16/FP32
- **动态调度**：根据训练阶段自动调整精度（如初期高精度、中期低精度、后期微调恢复）

训推第4个改进：多 Token 预测（MTP）架构演进

DeepSeekV3 在训练过程中引入了一个新方法，叫做 **多 Token 预测（Multi-Token Prediction，简称 MTP）**。

简单来说，就是让模型在学习的时候不只是猜下一个词，而是能一口气预测接下来多个词。

这就像我们背书一样，以前是一个字一个字地记，现在可以一段话一段话地理解记忆，效率更高，也更容易掌握整体意思。

为什么这样做更有用？

传统的语言模型在训练时，通常只关注“下一步该说什么”，也就是只预测一个词。但我们在实际说话或写作时，往往是一口气说出一整句话或者一段话。

MTP 让模型在训练阶段就学会这种“提前思考”的能力，相当于让它在学习时就能看到更远的地方。这样训练出来的模型，在回答问题、写文章、做推理等任务上表现得更好。

技术报告中的实验也验证了这一点——使用 MTP 后，模型在大多数测试任务中得分都有明显提升。

还能让生成更快？

更厉害的是，这个 MTP 不只是训练时有用，在模型生成文本的时候也能派上用场。

传统模型生成文字是“一个字一个字往外吐”，效率很低。而有了 MTP，模型可以一次预测多个词，就像批量输出一样，大大加快了生成速度。

多 Token 预测（MTP）核心流程图解

下面这张图展示了 MTP 在训练和推理过程中的核心流程：



Deepseek 训练 是从单 Token 预测 到 多 Token 预测演进

DeepSeek在语言模型训练目标上的技术演进路径，其核心是从传统的单Token预测（逐个预测）升级为多Token预测（并行预测）的范式转变。

具体差异如下：

1、单Token预测阶段

早期模型（如DeepSeek-V3初始设计）采用传统自回归方式，每次仅预测下一个Token，需迭代生成序列。

这种方式存在计算效率低（生成长文本需多次重复计算）和短期上下文依赖（难以捕捉长距离逻辑）的缺陷。

2、多Token预测（MTP）技术演进

DeepSeek通过以下创新实现突破：

- **并行预测架构**：模型通过多个输出头同时预测未来3-5个Token（如Head_1预测 $t+1$ ，Head_2预测 $t+2$ ），单次前向传播完成多步推理。
- **动态调整机制**：根据输入复杂度自适应调整预测Token数量（短文本精细预测，长序列宽泛前瞻）。
- **训练目标优化**：通过分层交叉熵损失联合优化多Token概率，增强长期连贯性。

这种演进使DeepSeek-R1的生成速度提升1.5倍，长文本困惑度降低18%，同时减少22%训练收敛时间。

总结一下多 Token 预测（MTP）

- **MTP 是什么？**

就是让模型在训练时一次性预测多个词，而不是只看下一步。

- **有什么好处？**

提升模型的理解和表达能力，还能在生成时加快速度。

- **有没有实验证明？**

有！官方的技术报告里做了对比实验，结果显示用了 MTP 的模型在多数任务上都更强。

参考资料（原始代码/图片）：

Benchmark (Metric)	# Shots	Small MoE	Small MoE	Large MoE	Large MoE
		Baseline	w/ MTP	Baseline	w/ MTP
# Activated Params (Inference)	-	2.4B	2.4B	20.9B	20.9B
# Total Params (Inference)	-	15.7B	15.7B	228.7B	228.7B
# Training Tokens	-	1.33T	1.33T	540B	540B
Pile-test (BPPB)	-	0.729	0.729	0.658	0.657
BBH (EM)	3-shot	39.0	41.4	70.0	70.7
MMLU (EM)	5-shot	50.0	53.3	67.5	66.6
DROP (F1)	1-shot	39.2	41.3	68.5	70.6
TriviaQA (EM)	5-shot	56.9	57.7	67.0	67.3
NaturalQuestions (EM)	5-shot	22.7	22.3	27.2	28.5
HumanEval (Pass@1)	0-shot	20.7	26.8	44.5	53.7
MBPP (Pass@1)	3-shot	35.8	36.8	61.6	62.2
GSM8K (EM)	8-shot	25.4	31.4	72.3	74.0
MATH (EM)	4-shot	10.7	12.6	38.6	39.8

CSDN 顺其自然~

推理部署方案的改进：预填充(Prefilling)和解码(Decoding)分离

DeepSeek-V3 是一个非常大的人工智能模型，参数量高达 **6710亿 (671B)** 。

你可以把它想象成一个超级大脑，里面装了非常多的知识和计算能力。

但正因为太大了，没法直接放在一台电脑上运行，所以需要一套聪明的部署方法，让它既能快速响应用户的问题，又能稳定运行。

一、推理过程拆分：预填充 + 解码

为了让这个大模型跑得更快更稳，它的推理过程被分成了两个阶段：

1、预填充阶段 (Prefill)

任务：理解用户输入的内容。比如你输入“请帮我写一首诗”，系统会先分析这句话的意思，准备好相关的背景知识。

类比：就像厨师看到菜单后，先准备好要用的食材，比如洗菜、切菜、备料。

2、解码阶段 (Decode)

任务：根据前面的理解，一步步生成回答内容。

类比：这一步就是厨师开始炒菜，一道一道地出成品，比如先炒青菜，再炒肉，最后装盘。

这两个阶段分开处理，可以让资源利用更高效，响应更快，同时也能处理更多用户的请求。

二、负载均衡策略：冗余专家 + 动态路由

因为模型太大，每个部分都可能成为瓶颈。

为了解决这个问题，系统用了两个关键技术：

- **冗余专家部署 (Redundant Experts)**
 - **意思**：把模型中的一些关键模块多复制几份，防止某个模块太忙导致卡顿。
 - **类比**：就像银行柜台，如果只有一个窗口排队会很长，多个窗口就能分流，加快效率。
- **动态路由策略 (Dynamic Routing)**
 - **意思**：系统会实时判断哪个模块空闲，自动把任务分配过去。
 - **类比**：就像导航软件，会帮你避开拥堵路段，走最快路线。

这两项技术配合使用，可以让整个系统的负载更加均衡，运行更稳定。

三、部署方式：跨机分布式推理

由于模型实在太太大，单台服务器根本装不下，所以必须采用**多台机器协同工作**的方式运行。

- 每台机器负责一部分计算任务，通过高速网络连接起来。
- 这样既能满足大模型的资源需求，又能保持响应速度。

核心流程图



总结

DeepSeek-V3 虽然参数量巨大，但通过以下手段实现了高效部署：

- 将推理过程分为 **预填充** 和 **解码** 两个阶段；
- 使用 **冗余专家** 和 **动态路由** 来平衡负载；
- 整体部署基于 **跨机分布式架构**，实现大规模并行计算。

这样既保证了响应速度，又提升了服务的稳定性与吞吐能力。

推理部署第一阶段：Prefill 阶段

阶段说明：Prompt 并行处理与 KV Cache 生成

这个阶段的核心任务是：

把用户输入的提示词 (Prompt) 快速处理成模型能直接使用的“记忆卡片”，也就是所谓的 KV Cache (Key-Value 缓存)。

你可以把它想象成提前整理好答案索引，这样后面回答问题时就能直接翻到对应页，不用每次都重新查一遍整本书。

整体部署结构

整个系统是以一个**最小部署单元**为单位运行的。

每个单元包含：

- **4个计算节点**
- 每个节点配备 **32块GPU**
- 所有GPU之间通过高速网络连接，可以高效协作完成任务

这种设计就像你安排了一个小团队，每个人分工明确，配合默契，效率自然就高。

注意力部分的并行策略

在处理 Prompt 的时候，有一项关键技术叫“注意力机制”，它决定了模型如何理解句子中词语之间的关系。

为了加快这部分处理速度，我们采用了以下几种并行方式：

1、4路张量并行（TP4）

模型中的大矩阵运算被拆分成4份，分别由不同的GPU来计算，最后再合并结果。

类比：就像四个人一起拼一幅大拼图，每人拼一块，最终合起来就是完整的画面。

2、序列并行（SP）

将输入的句子按长度切分，比如一句话有100个字，前25个字交给A GPU处理，中间25个交给B GPU，以此类推。

类比：像一群人接力读一本书，每人读一段，整体速度更快。

3、8路数据并行（DP8）

同时处理8组不同的 Prompt，每组分配给不同的GPU独立计算。

类比：就像8个学生各自做不同的题目，互不干扰，效率翻倍。

因为 TP 只用了4路，所以GPU之间的通信开销不大，整体效率比较高。

MoE 部分的并行策略

在模型中还有一个更复杂的模块叫做 **MoE（Mixture of Experts，专家混合）**，它的特点是根据输入内容动态选择最合适的“专家”来处理任务。

在这个阶段，我们使用了 **32路专家并行（EP32）**：

- 系统中有32个“专家”GPU
- 每个GPU负责处理特定类型的计算任务
- 根据输入内容，系统会自动挑选最适合的几个专家来协同处理

类比：就像你遇到一个问题，系统会从32位老师中挑出最擅长解答这个问题的几位来帮你解决问题。

核心流程图



参考代码

下面是一些用于配置并行策略的参数示例：

```
//示例配置（仅作参考）
tp_size = 4          # 张量并行数量
dp_size = 8          # 数据并行数量
ep_size = 32         # 专家并行数量
num_gpus_per_node = 32 # 每个节点GPU数量
```

这些参数可以帮助你实际部署时设置并行策略，确保系统高效运行。

推理部署第2阶段：Decoder 阶段

当你向大模型提问时，比如“太阳为什么发光？”，模型会一个字一个字地输出答案。

这个逐字生成的过程，叫做“**解码阶段**”。

可以把它想象成打字员一个字一个敲键盘，只不过这个“打字员”是AI模型。

一、解码阶段的部署结构

为了高效地完成这个“打字”任务，模型需要一个强大的“工作车间”来运行。

这个车间的最小配置是：

- 40个计算节点
- 320块GPU

你可以把它想象成一个工厂，每个GPU就像一台机器，负责一部分任务。整个车间协同工作，才能快速输出答案。

1. 注意力机制部分

这部分负责模型“记住”前面的内容，用到了三种并行技术：

- **TP4（张量并行4份）**：把一个大任务分成4份，同时处理。
- **SP（序列并行）**：把长句子拆成段，分段处理。
- **DP80（数据并行80份）**：把80个问题同时处理，提高效率。

2. MoE部分（专家机制）

这部分就像工厂里的“专业工人”，每个GPU只负责一个“专家”。

- 总共用了 **EP320（专家并行320份）**。
- 其中有64个GPU还额外负责“冗余专家”和“共享专家”，确保任务不漏掉、不重复。

可以把“专家”理解为工厂里的不同工种，比如有的负责焊接，有的负责组装。MoE就是根据任务需要，只调用相关的“专家”来工作，而不是所有人都一起上，节省了资源。

二、为什么DeepSeekV3训练成本这么低？

训练一个大模型是非常烧钱的，尤其是像DeepSeekV3这种超大规模模型。但它却能节省大量成本，主要靠两个方面：

- 模型架构设计
- 训练系统架构

这两方面配合得非常好，就像一辆车的发动机和变速箱完美匹配，才能跑得快又省油。

1. MLA机制：压缩记忆，节省空间

MLA（Multi-head Low-rank Attention）机制，是模型用来“记住”前面内容的一种方式。

- 模型在生成答案时，会记住前面已经说了什么，这部分叫“KV缓存”。
- 传统做法是减少记住的内容，而MLA是直接对这些内容进行“压缩”，就像把一张高清照片压缩成小图，占用更少内存。
- 这个技术对算法团队要求很高，需要很强的数学和工程能力。

2. FP8训练：用更少的精度做更高效的计算

FP8是一种低精度的数字格式，比FP16或FP32更“轻量”。

- 用FP8训练，就像用更轻的材料造车，跑得更快、更省油。
- DeepSeekV3首次在超大规模模型中成功使用FP8混合精度训练，这在工程上是一个很大的突破。
- 这意味着他们能在不牺牲效果的前提下，大幅节省GPU内存和计算时间。

3. MoE架构：只调用需要的“专家”

MoE（Mixture of Experts）是一种“按需调用”的架构。

- 模型运行时，只调用一部分“专家”来完成任务，而不是全部参数都参与计算。
- 相比传统的Dense模型（比如Qwen和Llama），MoE在训练和推理时都能节省大量计算资源。
- 但这也带来了挑战：如何调度这些“专家”？如何分配任务？如何处理专家之间的通信？这对工程团队提出了很高的要求。

三、核心流程图（Mermaid 图）

下面是一个简化的流程图，展示了DeepSeekV3训练和推理的核心流程：



四、总结

DeepSeekV3之所以能实现较低的训练成本，是多个技术点协同作用的结果：

- **MLA机制**：让KV缓存更小，节省内存；
- **FP8训练**：用更少的精度做高效的计算；
- **MoE架构**：只调用需要的专家，节省计算资源；

这些技术的背后，是算法和工程团队长期积累的结果，也体现了他们在大规模模型训练上的深厚实力。

DeepSeek 带个大家的思考

1.为什么DeepSeek这家中国AI公司这么受关注？

在硅谷，经常能看到一些公司做出很厉害的AI技术。

但这次不一样，这次做出突破的是一家**中国公司**——DeepSeek。

以前大家常说“美国搞创新，中国做应用”，但这次DeepSeek用实际行动打破了这种老观念，让人感到很振奋。

从他们最近发布的访谈和技术报告中，我们可以总结出几个关键原因：

1. 要做AI大模型，得有一支“全明星团队”

AI大模型不是一个人能搞定的，它需要一个**高水平、配合默契的团队**长期努力。就像一支职业篮球队，光有球星还不够，还需要教练、体能师、战术分析师等一起配合。

DeepSeek在这方面下了不少功夫，招揽了一批在AI领域有丰富经验的技术人才，形成了强大的研发能力。

2. 做大模型没有捷径，靠的是基本功和耐心

很多人以为做大模型靠的是什么“黑科技”或神秘算法，其实不是。真正关键的是**基础工作**：比如数据整理、模型调优、训练技巧等等。

这就像盖房子，地基打得牢，房子才能建得高。DeepSeek的成功，正是他们在这些“看不见的地方”下了苦功夫。

3. 不急着想赚钱，反而更容易做出成绩

很多初创公司一上来就想着怎么赚钱，结果反而限制了发展。DeepSeek选择了一条不同的路：**先专注技术打磨，不急于商业化**。

这种“轻装上阵”的策略，让他们可以大胆尝试各种技术路线，也更容易实现突破。

4. 国内AI发展 趋势？

在国内，AI更容易找到实际用得上的地方，比如智能客服、自动写文章、短视频生成等，这些产品用户能直接看到，企业也愿意花钱买。

不过在底层技术上，比如算法设计、训练方法这些方面，我们和国外相比还有一定差距。但随着国家和企业的投入加大，再加上像 DeepSeek 这样的国产大模型平台出现，差距正在慢慢缩小。

5. 大模型要发展，软硬件得一起上，生态建设很关键

想真正把大模型用起来，光靠软件是不行的，还得有合适的硬件支持。比如训练大模型需要高性能的GPU、TPU，推理时也需要高效的芯片和优化工具。

像 DeepSeek 这样的国产平台，不仅推动了技术进步，也让整个AI行业在开发、测试、部署等环节变得更高效、更便宜，加快了AI产品的更新迭代。

《LLM大模型学习圣经》 系列文章

尼恩架构团队，通过 梳理一个《LLM大模型学习圣经》帮助更多的人做LLM架构，拿到年薪100W，这个内容体系包括下面的内容：

- [《Python学习圣经：从0到1精通Python，打好AI基础》](#)
- [《LLM大模型学习圣经：从0到1吃透Transformer技术底座》](#)
- [《LangChain学习圣经：从0到1精通LLM大模型应用开发的基础框架》](#)
- 《LLM大模型学习圣经：从0到1精通RAG架构，基于LLM+RAG构建生产级企业知识库》
- [《SpringCloud + Python 混合微服务架构，打造AI分布式业务应用的技术底层》](#)
- 《LLM大模型学习圣经：从0到1吃透大模型的顶级架构》
- 《LLM 智能体 学习圣经：从0到1吃透 LLM 智能体 的架构 与实操》
- 《LLM 智能体 学习圣经：从0到1吃透 LLM 智能体 的 中台 架构 与实操》

更多顶级、优质文章，请参见 **技术自由圈** 公众号。

帮助大家，实现 技术自由。